

Ilutzim App — Development Setup Guide

Complete guide to running, configuring, and testing the Ilutzim App in a development environment.

Table of Contents

1. [Prerequisites](#)
 2. [Option A: Docker \(Recommended\)](#)
 3. [Option B: Local Development](#)
 4. [Telegram Bot Configuration](#)
 5. [Admin Dashboard Setup](#)
 6. [Environment Variables Reference](#)
 7. [Running Tests](#)
 8. [Useful Commands Cheat Sheet](#)
-

Prerequisites

Tool	Version	Purpose
Docker + Docker Compose	Latest	Running backend + database (Option A)
Python	3.12+	Backend API (Option B)
Node.js	18+	Frontend apps
PostgreSQL	16+	Database (Option B only)
Git	Latest	Version control

Option A: Docker (Recommended)

The fastest way to get the backend and database running.

1. Configure Environment

```
# The .env.docker file is already configured for Docker usage.  
# Review it and change the secrets before deploying:  
  
cat .env.docker
```

Key things to set in `.env.docker`:

- `SECRET_KEY` — Change to a random string
- `JWT_SECRET_KEY` — Change to a random string
- `TELEGRAM_BOT_TOKEN` — Your bot token (see [Telegram Bot Configuration](#))

2. Start the Stack

```
# Build and start all services (database + backend)
docker compose up -d --build

# View logs
docker compose logs -f backend
```

The entrypoint automatically:

1. Waits for PostgreSQL to be ready
2. Runs Alembic migrations (`alembic upgrade head`)
3. Seeds the default admin user
4. Starts the Uvicorn server

3. Verify It's Running

```
curl http://localhost:8000/health
```

4. Start the Frontends (Local)

The frontends run locally with hot-reload:

```
# Terminal 1 – Webapp (Guard submission UI, port 3000)
cd frontend/webapp
cp .env.example .env          # VITE_API_URL=http://localhost:8000
npm install
npm run dev

# Terminal 2 – Admin Dashboard (port 3001)
cd frontend/admin
cp .env.example .env          # VITE_API_URL=/api
npm install
npm run dev
```

5. Stop the Stack

```
docker compose down

# Stop and remove database data
docker compose down -v
```

Option B: Local Development

Run everything natively for hot-reload and easier debugging.

1. PostgreSQL Database

```
# Create the database
createdb ilutzim

# Or use psql:
psql -U postgres -c "CREATE DATABASE ilutzim;"
```

2. Backend

```
cd backend

# Create virtual environment
python -m venv .venv
source .venv/bin/activate      # Linux/Mac
# .venv\Scripts\activate      # Windows

# Install dependencies
pip install -r requirements.txt
pip install -r requirements-dev.txt # For running tests

# Configure environment
cp .env.example .env
```

Edit `backend/.env`:

```
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/ilutzim
DATABASE_URL_SYNC=postgresql://postgres:postgres@localhost:5432/ilutzim
TELEGRAM_BOT_TOKEN=your-bot-token-here
WEBAPP_URL=http://localhost:3000
ADMIN_DASHBOARD_URL=http://localhost:3001
ADMIN_API_KEY=dev-admin-api-key
JWT_SECRET_KEY=change-me-to-a-random-string
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=60
LOG_LEVEL=DEBUG
ENVIRONMENT=dev
CRON_WEEKLY_OPEN_DAY=sunday
CRON_WEEKLY_OPEN_HOUR=09:00
```

Run migrations and seed:

```
# Run database migrations
alembic upgrade head
```

```
# Seed the default admin user
python -m app.seed
```

Start the development server:

```
# With auto-reload
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

3. Frontend — Webapp (Guard Submission UI)

```
cd frontend/webapp

cp .env.example .env          # Creates: VITE_API_URL=http://localhost:8000
npm install                   # Starts on http://localhost:3000
npm run dev
```

This is the Telegram Web App interface that guards use to submit their weekly availability.

4. Frontend — Admin Dashboard

```
cd frontend/admin

cp .env.example .env          # Creates: VITE_API_URL=/api
npm install                   # Starts on http://localhost:3001
npm run dev
```

The admin dashboard proxies `/api` requests to `http://localhost:8000` automatically (configured in `vite.config.js`).

5. Access the Application

Service	URL
Backend API	http://localhost:8000
API Docs (Swagger)	http://localhost:8000/docs
Guard Webapp	http://localhost:3000
Admin Dashboard	http://localhost:3001

Telegram Bot Configuration

Creating the Bot

1. Open Telegram and search for **@BotFather**
2. Send `/newbot`
3. Choose a display name (e.g., "Ilutzim Scheduler")
4. Choose a username (must end in `bot`, e.g., `ilutzim_scheduler_bot`)
5. BotFather will give you a **bot token** that looks like:

```
123456789:ABCdefGhIjKlMnOpQrStUvWxYz
```

Configuring the Bot Token

For Docker: Edit `.env.docker`:

```
TELEGRAM_BOT_TOKEN=123456789:ABCdefGhIjKlMnOpQrStUvWxYz
```

For Local Development: Edit `backend/.env`:

```
TELEGRAM_BOT_TOKEN=123456789:ABCdefGhIjKlMnOpQrStUvWxYz
```

Note: Leave `TELEGRAM_BOT_TOKEN` empty to disable the bot. The API will still work without it.

Setting Up the Web App

To configure the bot's Web App (the guard submission form):

1. Message @BotFather again
2. Send `/mybots` → select your bot
3. Go to **Bot Settings** → **Menu Button** → set URL to your webapp URL
4. Or configure the webapp URL in the bot's inline keyboard

Setting Bot Commands (Optional)

In @BotFather, send `/setcommands` and paste:

```
start - Start the bot  
help - Show help
```

How the Bot Works

- The bot starts automatically when the backend starts (if `TELEGRAM_BOT_TOKEN` is set)
- **Cron jobs** run automatically:
 - Opens weekly submission on the configured day (`CRON_WEEKLY_OPEN_DAY` + `CRON_WEEKLY_OPEN_HOUR`)
 - Sends reminders at `REMINDER_HOUR` (default: 18:00)

- Guards interact with the bot to submit their availability via the embedded Web App

Admin Dashboard Setup

First Admin User

The first admin is created automatically (idempotent):

Docker: The entrypoint runs the seed script. Defaults from `.env.docker`:

```
Email:      admin@test.com
Password:   admin123
```

Local:

```
cd backend
python -m app.seed
```

You can override the credentials with environment variables:

```
SEED_ADMIN_EMAIL=admin@myorg.com \
SEED_ADMIN_PASSWORD=secure-password \
SEED_ADMIN_FULL_NAME="Admin Name" \
python -m app.seed
```

Logging In

1. Open `http://localhost:3001`
2. Enter the admin email and password
3. You'll receive a JWT token valid for 60 minutes (configurable)

Admin Features

The admin dashboard allows you to:

Page	Function
Guards	Manage guard users (add, edit, deactivate)
Weeks	Open/close weekly submission periods
Events	Create and manage schedule events
Submissions	View all guard submissions per week
Settings	Configure system-wide settings

Page	Function
Export	Export data to Excel files
Admins	Manage admin users and roles

Environment Variables Reference

Backend ([backend/.env](#))

Variable	Required	Default	Description
DATABASE_URL	✓	—	Async PostgreSQL connection string (postgresql+asyncpg://...)
DATABASE_URL_SYNC	×	—	Sync PostgreSQL connection string (for Alembic)
TELEGRAM_BOT_TOKEN	×	—	Telegram bot token from @BotFather. Leave empty to disable bot
WEBAPP_URL	✓	—	URL where the guard webapp is served (CORS)
ADMIN_DASHBOARD_URL	✓	—	URL where the admin dashboard is served (CORS)
ADMIN_API_KEY	✓	—	API key for admin endpoints
JWT_SECRET_KEY	✓	—	Secret for signing JWT tokens. Change in production!
JWT_ALGORITHM	×	HS256	JWT signing algorithm
JWT_ACCESS_TOKEN_EXPIRE_MINUTES	×	60	Token expiration time in minutes
LOG_LEVEL	×	INFO	Logging level: DEBUG , INFO , WARNING , ERROR
ENVIRONMENT	×	dev	Environment name: dev , production
CRON_WEEKLY_OPEN_DAY	×	sunday	Day of week to auto-open submissions
CRON_WEEKLY_OPEN_HOUR	×	09:00	Time to auto-open submissions (HH:MM)
REMINDER_HOUR	×	18	Hour to send closing reminders (0-23)

Seed Variables (for creating the first admin)

Variable	Default	Description
SEED_ADMIN_EMAIL	admin@test.com	Admin email address
SEED_ADMIN_PASSWORD	admin123	Admin password
SEED_ADMIN_FULL_NAME	Test Admin	Admin display name

Docker Environment (`.env.docker`)

Same variables as backend, but `DATABASE_URL` points to the Docker service name `db` instead of `localhost`:

```
DATABASE_URL=postgresql+asyncpg://ilutzim:ilutzim@db:5432/ilutzim
```

Frontend Webapp (`frontend/webapp/.env`)

Variable	Description
VITE_API_URL	Backend API URL, e.g., <code>http://localhost:8000</code>

Frontend Admin (`frontend/admin/.env`)

Variable	Description
VITE_API_URL	API base path (proxied by Vite), e.g., <code>/api</code>

Running Tests

Backend Unit/Integration Tests (Local)

```
cd backend
source .venv/bin/activate

# Run all tests
pytest -v

# Run a specific test file
pytest tests/test_health.py -v

# Run with coverage
pytest --cov=app -v
```

Frontend Tests


```
# Webapp
cd frontend/webapp
npm test

# Admin Dashboard
cd frontend/admin
npm test
```

E2E Tests (Docker)

Runs the full stack in Docker and executes end-to-end tests:

```
docker compose -f docker-compose.yml -f docker-compose.test.yml \
  up --build --abort-on-container-exit --exit-code-from e2e
```

This:

1. Starts PostgreSQL (in-memory, no persistent volume)
2. Builds and starts the backend
3. Seeds a test admin user
4. Runs `backend/tests/test_e2e.py` against the live backend
5. Exits with the test result code

Useful Commands Cheat Sheet

Docker

```
# Start all services
docker compose up -d --build

# View logs
docker compose logs -f

# Restart backend only
docker compose restart backend

# Rebuild after code changes
docker compose up -d --build backend

# Stop everything
docker compose down

# Stop and wipe database
docker compose down -v

# Run E2E tests
```

```
docker compose -f docker-compose.yml -f docker-compose.test.yml \  
up --build --abort-on-container-exit --exit-code-from e2e
```

Backend (Local)

```
cd backend  
source .venv/bin/activate  
  
# Start dev server with auto-reload  
uvicorn app.main:app --reload --port 8000  
  
# Run migrations  
alembic upgrade head  
  
# Create a new migration  
alembic revision --autogenerate -m "description"  
  
# Rollback one migration  
alembic downgrade -1  
  
# Seed admin user  
python -m app.seed  
  
# Run tests  
pytest -v  
  
# Check API health  
curl http://localhost:8000/health
```

Frontend

```
# Webapp (port 3000)  
cd frontend/webapp && npm install && npm run dev  
  
# Admin (port 3001)  
cd frontend/admin && npm install && npm run dev  
  
# Build for production  
cd frontend/webapp && npm run build  
cd frontend/admin && npm run build  
  
# Run tests  
cd frontend/webapp && npm test  
cd frontend/admin && npm test
```

Database

```
# Connect to PostgreSQL (Docker)
docker compose exec db psql -U ilutzim -d ilutzim

# Connect to PostgreSQL (local)
psql -U postgres -d ilutzim

# Reset database (Docker)
docker compose down -v
docker compose up -d --build
```

Quick Start (TL;DR)

```
# 1. Clone the repo
git clone <repo-url> && cd ilutzim_app

# 2. Start backend + database
docker compose up -d --build

# 3. Start frontends (in separate terminals)
cd frontend/webapp && cp .env.example .env && npm install && npm run dev
cd frontend/admin && cp .env.example .env && npm install && npm run dev

# 4. Open your browser
#   Admin: http://localhost:3001 (admin@test.com / admin123)
#   Webapp: http://localhost:3000
#   API Docs: http://localhost:8000/docs
```